КРИПТОГРАФІЯ
Комп’ютерний практикум №2
Криптоаналіз шифру Віженера

Виконали: Сімутін Владислав ФБ-42
Сирота Володимир ФБ-42

Мета роботи

Засвоєння методів частотного криптоаналізу; здобуття навичок роботи та аналізу потокових шифрів гамування адитивного типу на прикладі шифру Віженера.

1. Опис вхідних даних та правил їх підготовки

Для дослідження шифру використано російськомовний художній уривок (казка про Їжачка), що зберігається у файлі text.txt. Алфавіт роботи містить m = 32 літери: «абвгдежзийклмнопрстуфхцчшщъыьэюя». Правила підготовки тексту (скрипт prepared_text.py): усі літери переводяться в нижній регістр; літера «ё» замінюється на «е»; видаляються всі символи, що не належать алфавіту (пробіли, розділові знаки, лапки, тире, перенесення рядків). Результат записується у prepared.txt і містить 937 літер.

```python
import re

INPUT_FILE = "text.txt"

OUTPUT_FILE = "prepared.txt"

ALPHABET = "абвгдежзийклмнопрстуфхцчшщъыьэюя"

with open(INPUT_FILE, "r", encoding="utf-8") as file:

    text = file.read()

text = text.lower()

text = text.replace("ё", "е")

text = "".join(

    char for char in text

    if char in ALPHABET

)

with open(OUTPUT_FILE, "w", encoding="utf-8") as file:

    file.write(text)
```

Шифртекст варіанта 4 (файл var4.txt) аналогічно очищується скриптом var4.py: переводиться в нижній регістр, залишаються лише літери алфавіту, результат записується у var4r.txt. Після очищення довжина шифртексту n = 8061 літер.

2. Короткий опис програмних засобів

Таблиця 1 – Реалізовані програмні засоби

Файл	Призначення
prepared_text.py, var4.py	Підготовка відкритого тексту та шифртексту варіанта (фільтрація за алфавітом)

Файл	Призначення
part1.py	Шифрування prepared.txt шифром Віженера ключами довжини 2, 3, 4, 5 та 15 → enc2…enc15.txt
index.py	Обчислення індексу відповідності для відкритого тексту та всіх шифртекстів
part2-1.py	Статистика збігів D_r для r = 2…30
part2-3.py	Середній індекс відповідності блоків Y_i для r = 1…30
part2-2.py	Частоти літер у блоках Y_i для кандидатів r = 11, 13, 14, 26, 28
part3-1.py, part3-2.py, part3-3.py	Відновлення ключа, припускаючи, що найчастіша літера блока відповідає «о», «а» або «е»
dec.py	Розшифрування var4r.txt знайденим ключем
part4.py	Експеримент із помилкою в одному символі ключа

3. Дослідження шифру Віженера для ключів різної довжини

Відкритий текст зашифровано шифром y_i = (x_i + k_(i mod r)) mod 32 п’ятьма ключами: «то» (r = 2), «где» (r = 3), «ежик» (r = 4), «слева» (r = 5) та «тольконачинался» (r = 15). Результати збережено у файлах enc2.txt, enc3.txt, enc4.txt, enc5.txt, enc15.txt.

```python
INPUT_FILE = "prepared.txt"

ALPHABET = "абвгдежзийклмнопрстуфхцчшщъыьэюя"

KEYS = [
    "то",
    "где",
    "ежик",
    "слева",
    "тольконачинался"
]

with open(INPUT_FILE, "r", encoding="utf-8") as file:
    text = file.read()

for key in KEYS:
    encrypted = ""

    for i, char in enumerate(text):
        x = ALPHABET.index(char)
        k = ALPHABET.index(key[i % len(key)])
        y = (x + k) % len(ALPHABET)
        encrypted += ALPHABET[y]

    output_file = f"enc{len(key)}.txt"

    with open(output_file, "w", encoding="utf-8") as file:
        file.write(encrypted)

    print(f"{output_file}")
```

```
enc2.txt
enc3.txt
enc4.txt
enc5.txt
enc15.txt
```

Індекс відповідності обчислюється за формулою I(Y) = Σ N_t(N_t − 1) / (n(n − 1)) (скрипт index.py).

```python
from collections import Counter

FILES = [
    "prepared.txt",
    "enc2.txt",
    "enc3.txt",
    "enc4.txt",
    "enc5.txt",
    "enc15.txt"
]

def coincidence_index(text):
    n = len(text)
    frequencies = Counter(text)

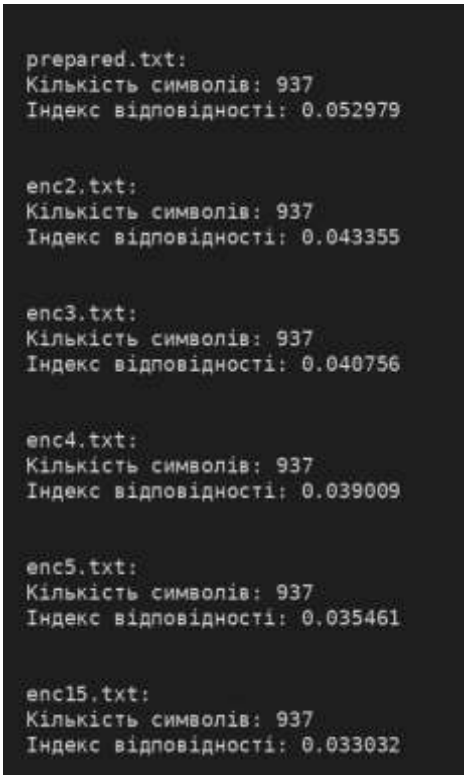
    numerator = sum(count * (count - 1) for count in frequencies.values())
    denominator = n * (n - 1)

    return numerator / denominator

for filename in FILES:
    with open(filename, "r", encoding="utf-8") as file:
        text = file.read().strip()

    index = coincidence_index(text)

    print(f"\n{filename}:")
    print(f"Кількість символів: {len(text)}")
    print(f"Індекс відповідності: {index:.6f}")
    print()
```



Таблиця 2 – Індекси відповідності відкритого тексту та шифртекстів (n = 937)

Текст	Довжина ключа r	Ключ	I(Y)
Відкритий текст	–	–	0,052979
enc2	2	то	0,043355

Текст	Довжина ключа r	Ключ	I(Y)
enc3	3	где	0,040756
enc4	4	ежик	0,039009
enc5	5	слева	0,035461
enc15	15	тольконачинался	0,033032

Індекс відповідності відкритого тексту (≈ 0,053) близький до теоретичного значення для російської мови (≈ 0,055) і значно перевищує значення 1/m = 1/32 ≈ 0,0313, яке відповідає рівноймовірному алфавіту. Зі збільшенням довжини ключа індекс монотонно спадає та наближається до 1/32: при r = 15 він становить 0,0330. Причина в тому, що при довшому ключі літери кожного значення x однієї частоти розподіляються між більшою кількістю різних зсувів, і розподіл літер шифртексту стає майже рівномірним. Для короткого ключа (r = 2) шифртекст ще зберігає помітну частину статистики мови.

4. Визначення довжини ключа шифртексту варіанта 4

Загальний індекс відповідності шифртексту var4r.txt (n = 8061) становить I = 0,032630, що дуже близько до 1/32 = 0,03125. Це вказує на те, що ключ не є коротким, і прості підходи (другий алгоритм для r ≤ 5) тут неефективні. Тому застосовано два методи, описані в методичних відомостях.

4.1. Метод статистики збігів D_r

Для кожного r = 2…30 підраховується D_r – кількість позицій i, для яких y_i = y_(i+r). Для відстаней, кратних періоду, збіги трапляються з імовірністю $\Sigma p_t^2 \approx 0{,}054$, для інших – ≈ 1/32.

```python
INPUT_FILE = "var4r.txt"

with open(INPUT_FILE, "r", encoding="utf-8") as file:
    text = file.read().strip()

n = len(text)

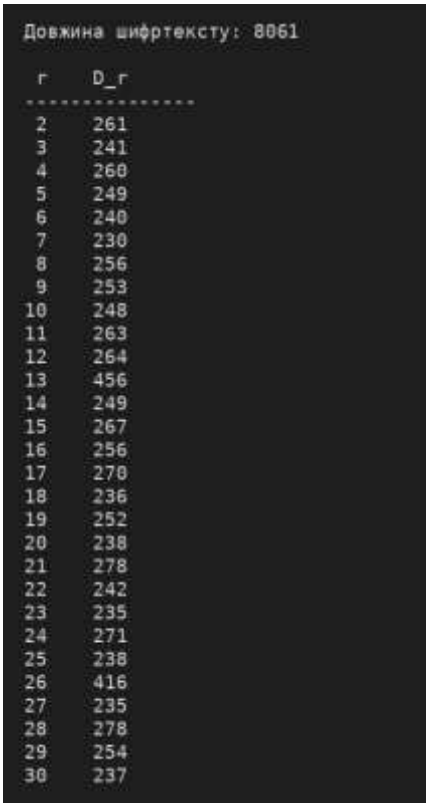
print(f"Довжина шифртексту: {n}")
print()

print(" r    D_r")
print("-" * 15)

for r in range(2, 31):
    matches = 0

    for i in range(n - r):
        if text[i] == text[i + r]:
            matches += 1

    print(f"{r:2}    {matches}")
```



```
Довжина шифртексту: 8061

 r    D_r
---------------
 2    261
 3    241
 4    260
 5    249
 6    240
 7    230
 8    256
 9    253
10    248
11    263
12    264
13    456
14    249
15    267
16    256
17    270
18    236
19    252
20    238
21    278
22    242
23    235
24    271
25    238
26    416
27    235
28    278
29    254
30    237
```



Для переважної більшості r значення D_r лежать у межах 230–278 (середнє для некратних 13 – 251,9, що відповідає очікуваному ≈ n/32 ≈ 252). Виразно виділяються два піки: **D_13 = 456** та **D_26 = 416**. Очікуване значення для кратних періоду ≈ (n − r)·0,054 ≈ 435, що узгоджується зі спостережуваним. Пік на r = 26 = 2·13 є кратним першому, тобто підтверджує, що істинний період дорівнює 13 (r = 39 виходить за межі перевіреного діапазону).

4.2. Метод середнього індексу відповідності блоків

Шифртекст розбивається на r блоків Y_i = y_i, y_(i+r), y_(i+2r), …; для кожного обчислюється I(Y_i) та береться середнє значення. Якщо r дорівнює справжньому періоду (або кратне йому), кожен блок є шифром Цезаря, і індекс близький до індексу мови; інакше – до 1/32.

```python
from collections import Counter

INPUT_FILE = "var4r.txt"

with open(INPUT_FILE, "r", encoding="utf-8") as file:
    text = file.read().strip()

def coincidence_index(t):
    n = len(t)
    f = Counter(t)
    return sum(c * (c - 1) for c in f.values()) / (n * (n - 1))

print(f"Довжина шифртексту: {len(text)}")
print(f"Індекс відповідності всього шифртексту: {coincidence_index(text):.6f}")
print()
print(" r    середній I(Y_i)")
print("-" * 22)

for r in range(1, 31):
    blocks = [text[i::r] for i in range(r)]
    avg = sum(coincidence_index(b) for b in blocks) / r
    print(f"{r:2}    {avg:.6f}")
```

```
Довжина шифртексту: 8061
Індекс відповідності всього шифртексту: 0.032630

 r    середній I(Y_i)
----------------------
 1    0.032630
 2    0.032605
 3    0.032577
 4    0.032651
 5    0.032535
 6    0.032560
 7    0.032718
 8    0.032692
 9    0.032514
10    0.032518
11    0.032714
12    0.032635
13    0.054069
14    0.032637
15    0.032436
16    0.032675
17    0.032683
18    0.032569
19    0.032665
20    0.032507
21    0.032769
22    0.032516
23    0.032672
24    0.032639
25    0.032509
26    0.053855
27    0.032348
28    0.032491
29    0.032363
30    0.032398
```



Для всіх r, крім 13 та 26, середнє значення становить 0,0323–0,0327, тобто відповідає рівноймовірному алфавіту. Для r = 13 середній індекс різко зростає до 0,0541, для r = 26 – до 0,0539, що співпадає з індексом відповідності російської мови. Обидва методи дають однаковий результат: довжина ключа r = 13.

5. Аналіз альтернативних кандидатів на довжину ключа

Для додаткової перевірки досліджено кандидатів r = 11, 13, 14, 26 та 28 (скрипт part2-2.py). Вибір обґрунтовано так: r = 13 та r = 26 – піки статистики D_r; r = 11 та r = 14 – сусідні значення, що слугують контролем (при невірному r блоки мають бути «плоскими»); r = 28 – одне з найбільших фонових значень D_r (278, поряд з r = 21), тобто «найнебезпечніший» хибний кандидат. Для кожного блока знаходилась частка найчастішої літери.

```python
from collections import Counter

INPUT_FILE = "var4r.txt"

CANDIDATES = [11, 13, 14, 26, 28]

with open(INPUT_FILE, "r", encoding="utf-8") as file:
    text = file.read().strip()

for r in CANDIDATES:
    print("=" * 50)
    print(f"ДОВЖИНА КЛЮЧА: {r}")
    print("=" * 50)

    for i in range(r):
        block = text[i::r]
        frequencies = Counter(block)
        most_common = frequencies.most_common(3)

        print(
            f"Блок {i + 1:2}: "
            f"{most_common[0][0]}={most_common[0][1]}, "
            f"{most_common[1][0]}={most_common[1][1]}, "
            f"{most_common[2][0]}={most_common[2][1]}"
        )

    print()
```

Таблиця 3 – Частка найчастішої літери в блоках Y_i

r	Довжина блока	Середня частка	Мін.	Макс.	Висновок
11	733	0,055	0,049	0,063	хибний
13	621	0,108	0,090	0,126	істинний
14	576	0,056	0,049	0,061	хибний
26	311	0,110	0,084	0,138	кратний 13
28	288	0,058	0,045	0,076	хибний

Для r = 13 у всіх блоках одна літера домінує (56–78 поя в із ≈ 621, 9–13 %), що характерно для шифру Цезаря від мови з домінуванням літери «о». Для r = 11, 14, 28 жодна літера в блоках не домінує виразно (середня частка найчастішої літери ≈ 5–6 %, а в окремих блоках її перевага над другою за частотою мізерна), як і має бути для суміші різних зсувів. Для r = 26 вираженість така ж, як для 13, але блоки вдвічі коротші (≈ 311), тому статистика шумніша; крім того, у 10 із перших 13 блоків при r = 26 домінантні літери збігаються з відповідними блоками при r = 13 (с, ь, ъ, ы, о, к, т, б, ъ, о), тобто ключ довжини 26 – це двічі повторений ключ довжини 13.

6. Відновлення ключа

Для кожного з 13 блоків визначено найчастішу літеру y* і обчислено k = (y* − x*) mod 32, де x* – припущена найімовірніша літера відкритого тексту. Спочатку x* = «о» (найчастіша літера російської мови; скрипт part3-1.py).

```python
from collections import Counter

INPUT_FILE = "var4r.txt"

ALPHABET = "абвгдежзийклмнопрстуфхцчшщъыьэюя"

KEY_LENGTH = 13

with open(INPUT_FILE, "r", encoding="utf-8") as file:
    text = file.read().strip()

key = ""

print(f"Довжина шифртексту: {len(text)}")
print()

for i in range(KEY_LENGTH):
    block = text[i::KEY_LENGTH]
    frequencies = Counter(block)

    most_common_char = frequencies.most_common(1)[0][0]

    cipher_index = ALPHABET.index(most_common_char)
    plain_index = ALPHABET.index("о")
    key_index = (cipher_index - plain_index) % len(ALPHABET)

    key_char = ALPHABET[key_index]
    key += key_char

    print(
        f"Блок {i + 1}: "
        f"{most_common_char} = {frequencies[most_common_char]} "
        f"-> ключ: {key_char}"
    )

print()
print("Попередній ключ:", key)
```

```
Довжина шифртексту: 8061

Блок 1: с = 78 -> ключ: г
Блок 2: ю = 56 -> ключ: р
Блок 3: ь = 71 -> ключ: о
Блок 4: ъ = 70 -> ключ: м
Блок 5: ы = 56 -> ключ: н
Блок 6: ш = 62 -> ключ: к
Блок 7: о = 64 -> ключ: а
Блок 8: р = 62 -> ключ: в
Блок 9: к = 70 -> ключ: ь
Блок 10: т = 62 -> ключ: д
Блок 11: б = 78 -> ключ: у
Блок 12: ъ = 68 -> ключ: м
Блок 13: о = 72 -> ключ: а

Попередній ключ: громнкавьдума
```

Попередній ключ: «громнкавьдума». Він частково осмислений («гром…», «…вь…ма»), що свідчить про правильність більшості символів. У блоках 5, 7, 9, 11 літера «о» не є найчастішою, тому для них перевірено друге та третє за ймовірністю припущення – «а» і «е» (скрипти part3-2.py та part3-3.py, змінюється лише один рядок):

```
24    plain_index = ALPHABET.index("о")
```

```
24    plain_index = ALPHABET.index("а")
```

```
24    plain_index = ALPHABET.index("е")
```

```
Довжина шифртексту: 8061

Блок 1: с = 78 -> ключ: с
Блок 2: ю = 56 -> ключ: ю
Блок 3: ь = 71 -> ключ: ь
Блок 4: ъ = 70 -> ключ: ъ
Блок 5: ы = 56 -> ключ: ы
Блок 6: ш = 62 -> ключ: ш
Блок 7: о = 64 -> ключ: о
Блок 8: р = 62 -> ключ: р
Блок 9: к = 70 -> ключ: к
Блок 10: т = 62 -> ключ: т
Блок 11: б = 78 -> ключ: б
Блок 12: ъ = 68 -> ключ: ъ
Блок 13: о = 72 -> ключ: о

Попередній ключ: сюъъышорктбъо
```

```
Довжина шифртексту: 8061

Блок 1: с = 78 -> ключ: м
Блок 2: ю = 56 -> ключ: щ
Блок 3: ь = 71 -> ключ: ч
Блок 4: ъ = 70 -> ключ: х
Блок 5: ы = 56 -> ключ: ц
Блок 6: ш = 62 -> ключ: у
Блок 7: о = 64 -> ключ: й
Блок 8: р = 62 -> ключ: л
Блок 9: к = 70 -> ключ: е
Блок 10: т = 62 -> ключ: н
Блок 11: б = 78 -> ключ: ь
Блок 12: ъ = 68 -> ключ: х
Блок 13: о = 72 -> ключ: й

Попередній ключ: мщчхцуйленьхй
```

Таблиця 4 – Проміжні результати відновлення ключа (n_i – кількість появ найчастішої літери блока)

Блок	y*	n_i	k для «о»	k для «а»	k для «е»	Обраний символ	Гіпотеза
1	с	78	г	с	м	г	о
2	ю	56	р	ю	щ	р	о
3	ь	71	о	ь	ч	о	о
4	ъ	70	м	ъ	х	м	о
5	ы	56	н	ы	ц	ы	а
6	ш	62	к	ш	у	к	о
7	о	64	а	о	й	о	а
8	р	62	в	р	л	в	о
9	к	70	ь	к	е	е	е

Блок	y*	n_i	k для «о»	k для «а»	k для «е»	Обраний символ	Гіпотеза
10	т	62	д	т	н	д	о
11	б	78	у	б	ь	ь	е
12	ъ	68	м	ъ	х	м	о
13	о	72	а	о	й	а	о

Для 9 блоків із 13 (1, 2, 3, 4, 6, 8, 10, 12, 13) найчастіша літера відповідає «о»; для блоків 5 і 7 – літері «а», для блоків 9 і 11 – літері «е». Правильність вибору підтверджено тим, що ключ «громыковедьма» є осмисленою фразою (прізвище та слово «ведьма»), а розшифрований текст читається. Прізвище в ключі та слово дозволили виправити 4 помилкові символи попереднього ключа (н→ы, а→о, ь→е, у→ь).

7. Розшифрування шифртексту та перевірка результату

Знайдений ключ: «громыковедьма» (13 літер). Розшифрування виконано за формулою x_i = (y_i − k_(i mod 13)) mod 32 (скрипт dec.py).

```python
INPUT_FILE = "var4r.txt"
OUTPUT_FILE = "decrypted.txt"

ALPHABET = "абвгдежзийклмнопрстуфхцчшщъыьэюя"
KEY = "громыковедьма"

with open(INPUT_FILE, "r", encoding="utf-8") as file:
    text = file.read().strip()

decrypted = ""

for i, char in enumerate(text):
    y = ALPHABET.index(char)
    k = ALPHABET.index(KEY[i % len(KEY)])
    x = (y - k) % len(ALPHABET)
    decrypted += ALPHABET[x]


print("Ключ:", KEY)
print("Довжина тексту:", len(decrypted))
with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
    f.write(decrypted)
print("Результат записано у:", OUTPUT_FILE)
print()
print(decrypted[:500])
```

```
Ключ: громыковедьма
Довжина тексту: 8061
Результат записано у: decrypted.txt
старминскаяшколачародеевпифийитравницфакультеттеоретическойипрактическоймагиикафедрамаговп
```

Початок відкритого тексту: «старминская школа чародеев, пифий и травниц. Факультет теоретической и практической магии. Кафедра магов-практиков. Часть первая. Социальный уклад, быт, нравы вампирьей общины…». Текст змістовний, містить слова та словосполучення російської

мови на всій довжині (наприклад, кінець: «…десятиаршинная статуя, которого украшала актовый зал высшей школы…»).

Перевірка правильності: (1) отриманий текст осмислений на всій довжині 8061 літер; (2) індекс відповідності розшифрованого тексту дорівнює 0,0541 і відповідає значенню для російської мови; (3) частоти літер типові для російської (о – 824, е – 703, а – 696, и – 545, н – 504); (4) сам ключ «громыковедьма» є змістовною фразою, як зазначено в методичних вказівках.

8. Дослідження впливу помилки у ключі

Прогноз до експерименту. У шифрі Віженера кожна літера ключа використовується лише для літер відкритого тексту з позиціями $i \equiv j \pmod{13}$ і не впливає на інші, а шифрування є адитивним, без розсіювання. Тому заміна одного символу ключа (5-й символ «ы» → «н») має спотворити лише кожну 13-ту літеру (приблизно $8061/13 \approx 620$ літер, 7,7 %), починаючи з позиції 4 (нумерація з 0); решта 92,3 % тексту залишиться правильною. Кожна спотворена літера має бути зміщена на одну й ту саму сталу: різниця положень «н» (14) та «ы» (28) за модулем 32 дорівнює 18, тобто розшифрована літера буде на 18 позицій меншою за правильну (mod 32).

```python
ALPHABET = "абвгдежзийклмнопрстуфхцчшщъыьэюя"
M = len(ALPHABET)

INPUT_FILE = "var4r.txt"
KEY = "громыковедьма"
POS = 4                          # номер символу ключа (з 0), який змінюємо
NEW = "н"                        # ы -> н

with open(INPUT_FILE, "r", encoding="utf-8") as file:
    text = file.read().strip()

def decrypt(t, key):
    return "".join(
        ALPHABET[(ALPHABET.index(c) - ALPHABET.index(key[i % len(key)])) % M]
        for i, c in enumerate(t)
    )

bad_key = KEY[:POS] + NEW + KEY[POS + 1:]
good = decrypt(text, KEY)
bad = decrypt(text, bad_key)

diff = [i for i in range(len(good)) if good[i] != bad[i]]
shift = (ALPHABET.index(NEW) - ALPHABET.index(KEY[POS])) % M

print("Правильний ключ:", KEY)
print("Хибний ключ:    ", bad_key)
print(f"Зсув у ключі: {shift}")
print(f"Всього літер: {len(good)}, змінилось: {len(diff)} ({len(diff)/len(good):.2%})")
print(f"Теоретично 1/13 = {1/13:.2%}")
print("Перші позиції змін:", diff[:8])
print("Різниці позицій:", sorted({diff[i+1]-diff[i] for i in range(len(diff)-1)}))
print()
print("Правильно:", good[:65])
print("З помилкою:", bad[:65])
print()
print("Заміна на позиціях змін (перші 5):")
for i in diff[:5]:
    d = (ALPHABET.index(good[i]) - ALPHABET.index(bad[i])) % M
    print(f"  позиція {i}: {good[i]} -> {bad[i]}  (зсув {d})")
```

```
Правильний ключ: громыковедьма
Хибний ключ:     громнковедьма
Зсув у ключі: 18
Всього літер: 8061, змінилось: 620 (7.69%)
Теоретично 1/13 = 7.69%
Перші позиції змін: [4, 17, 30, 43, 56, 69, 82, 95]
Різниці позицій: [13]

Правильно: старминскаяшколачародеевпифийитравницфакультеттеоретическойипракт
З помилкою: старъинскаяшколачородеевпифийиаравницфакульаеттеоретичесшойипракт

Заміна на позиціях змін (перші 5):
  позиція 4: м -> ъ  (зсув 18)
  позиція 17: а -> о  (зсув 18)
  позиція 30: т -> а  (зсув 18)
  позиція 43: т -> а  (зсув 18)
  позиція 56: к -> ш  (зсув 18)
```

Результат повністю підтвердив прогноз: змінилось 620 літер із 8061 (7,69 % = 1/13); зміни розташовані на позиціях 4, 17, 30, 43, 56, … з незмінним кроком 13; усі змінені літери зміщені на однакову величину 18 (м→ъ, а→о, т→а, к→ш тощо). Текст залишається впізнаваним: «старъинская школа чородеев пифийи аравниц факульает теоретичесшой…» – помилки періодичні й легко виправляються за змістом. Це демонструє відсутність дифузії в шифрі Віженера й те, що кожен символ ключа можна відновлювати незалежно.

9. Відповіді на питання для аналізу результатів

1. Як змінювався індекс відповідності при ключах різної довжини? Він монотонно спадав від 0,0530 (відкритий текст) через 0,0434 (r = 2), 0,0408 (r = 3), 0,0390 (r = 4), 0,0355 (r = 5) до 0,0330 (r = 15), наближаючись до 1/32 ≈ 0,0313. Чим довший ключ, тим більше різних шифрів Цезаря «змішано» в шифртексті, і розподіл літер стає ближчим до рівномірного.

2. За якими ознаками визначено довжину ключа? За двома піками статистики збігів D_13 = 456 та D_26 = 416 (фон ≈ 252) і за зростанням середнього індексу відповідності блоків до 0,0541 при r = 13 (та 0,0539 при r = 26) проти ≈ 0,0326 для інших r.

3. Які альтернативні кандидати досліджено? r = 26 (кратний пік), r = 11 та 14 (сусіди-контроль) і r = 28 (найбільше фонове значення D_r). Для 11, 14, 28 частка найчастішої літери в блоках ≈ 5–6 %, блоки «плоскі»; для 26 – така ж, як для 13, але блоки коротші; ключ довжини 26 дублює ключ довжини 13.

4. Яка статистика використовувалась для символів ключа? Частоти літер у кожному з 13 блоків: найчастішу літеру блока зіставляли з «о», а де це не підтверджувалось – з «а» та «е» (див. табл. 4).

5. Як перевірити ключ, не знаючи відкритого тексту? За змістовністю розшифрованого тексту, за близькістю індексу відповідності результату до 0,055, за частотами літер, близькими до мовних, та за змістовністю самого ключа.

6. Як змінився результат після помилки в одному символі? Спотворилась кожна 13-та літера (620 із 8061), усі на однакове зміщення 18; решта тексту не змінилась.

7. Які етапи найбільше залежать від обсягу шифртексту? Визначення довжини ключа (D_r та індекс блоків потребують достатньо довгих блоків; при великих r блоки коротшають і статистика шумить) та вибір символів ключа за частотами в блоках: у блоці довжиною ≈ 620 літер частотний максимум ще надійний, у коротшому – ні.

8. Які результати найкраще демонструють слабкість шифру? Різкі піки D_13 і середнього індексу блоків на істинному періоді; відновлення ключа з частот у блоках; а також експеримент із помилкою, що показує незалежність частин ключа та відсутність дифузії.

Висновки

У роботі досліджено шифр Віженера та методи його криптоаналізу. Показано, що індекс відповідності зашифрованого тексту спадає з ростом довжини ключа до значення 1/32 для рівноймовірного алфавіту. Для шифртексту варіанта 4 (n = 8061) довжину ключа визначено двома методами – за статистикою збігів D_r та за середнім індексом відповідності блоків: r = 13 (D_13 = 456, середній I = 0,0541; r = 26 є кратним). Перевірено альтернативні кандидати 11, 14, 26, 28. Ключ відновлено частотним методом:

«громыковедьма»; за допомогою гіпотез «о», «а», «е» усунуто помилки в 4 символах ключа. Розшифрований текст змістовний, його індекс відповідності 0,0541. Експеримент із помилкою в ключі підтвердив прогноз: спотворюється лише кожна 13-та літера на сталу величину. Отже, шифр Віженера з періодичним ключем не стійкий до частотного аналізу, особливо при довжині шифртексту, значно більшій за довжину ключа.